DRF

- serializer序列化器
  - 相比之前解决了痛点?
  - 1.序列化器定义和校验顺序
  - 2.序列化器默认提供了CU的方法
  - 模型序列化器
    - 手动定义序列化器 麻烦。
    - 怎么优化，引入ModelSerializer
      - 怎么使用?
        - 1.继承ModelSerializer
        - 2.需要在 内部Mate类中指明一些依赖信息
          - model = Projects
          - # fields = '__all__' # 序列化输出所有字段信息
            # fields = ('id', 'name', 'leader') # 输出指定字段
            exclude = ('create_time', 'update_time') # 排除序列化输出字段
          - read_only_fields = ('is_build', 'tester')
        - 3.对自动生成的序列化器中字段不满意，处理方式
          - 1.重写字段，自己定义即可，优先级大于自动生成的
          - 2.能用就用，不能用改一改了再用
          - 在 Mate类中 指明属性
            # 对不满足需求的字段信息，进行重新定义
            # key:需要修改的字段名 value:修改的值
            extra_kwargs = {'name': {'max_length': 3}}
        - 4.特殊场景如：*反序列化的时候要验证邮箱。
          序列化输出的时候 要输出token。
          这些在模型类中都没有字段，自动生成的序列化器也没提供
          - 1.要在序列化器中定义这些需要使用的字段
          - 2.如要是要反序列化验证的 如验证码等.不需要存数据库
            数据验证通过了，就删除掉，因为模型类中没字段，存数据库会抛出异常
          - 3.如果是要序列化时候输出的 如token
            数据验证通过，先存数据库，在添加动态属性，给obj.token = 赋值
            这时候序列化的数据就包含token
          - 4.思路有了 操作有哪些注意事项：
            - 1.新增加的字段，要加入到序列化器中
              fields = ('id', 'name', 'leader') 输出指定方式要添加
            - fields = '__all__' or exclude = ('create_time', 'update_time') 不用添加
    - 解决了什么问题? 提高效率
      - 1.根据模型 ==》 自动生成序列化器
      - 2.模型类字段中 有约束字段，自动添加约束验证点 比如唯一约束
        unique=True
        validators=[<UniqueValidator(queryset=Projects.objects.all())>]
      - 3.模型字段有 主键 创建时间，修改时间等 这些特殊字段，生成的序列化器中，会自动增加read_only=True
      - 4.是否可以传null 和是否可以空 模型类字段怎么定义，就根据模型类字段自动生成
      - 5.模型类中字段 提供了默认值，序列化器内将生成required=False 不是必填项
      - 6.默认提供了 create 和 update 方法
- View优化
  - APIView
    - 解决了什么问题?
    - 1.相比于django中的view,提供了解析器和渲染器
    - 2.根据不同的解析器，取值都是request.data
    - 3.请求和响应 重新封装成了 Request和Response
    - 4.提供了认证，授权，限流等
  - GenericAPIView
    - 主要来处理 分页排序过滤
      - 过滤
        - queryset = self.filter_queryset(queryset=self.get_queryset())
      - 分页
        - page = self.paginate_queryset(queryset=queryset)
        - serializer = self.get_serializer(instance=page, many=True)
        - return self.get_paginated_response(data=serializer.data)
    - 每个视图类都需要使用不同的序列化器和查询集，将其抽离出来，使用get_queryset 和 get_serializer 获取
- mixin扩展类
  - 解决了什么问题?
  - CURDL方法大致流程都是类似的，所以就封装了起来
  - 使用方式：继承过来即可
  - 在对应方法里面 调用父类的 实现方法